SENTINEL AI — Technical Document

Real-time fraud detection, intervention and investigation control tower

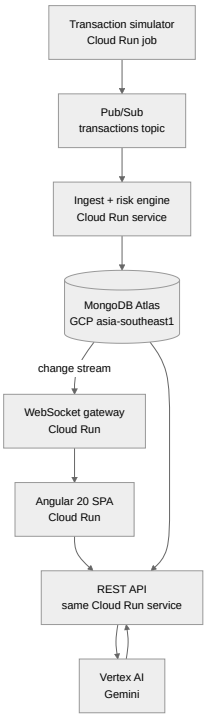
Mohammed Ilyas · first issued 20 Sep 2026 · updated 24 Sep 2026 (post-review) v1.2

> Companion to the SENTINEL AI Master Document. This one covers how it is built; the master covers what and why.

---

1. System architecture

Four planes. Each has a single owner and a single deployable.



Plane	Responsibility	Runtime
Ingest	Emit and transport transaction events	Cloud Run job, Pub/Sub
Compute	Score, gate, persist, fan out	Cloud Run service, MongoDB Atlas
Intelligence	Explain, correlate, summarise, report	Vertex AI Gemini, called from the same service
Presentation	Stream, render, intervene	Angular 20 SPA, Web Worker, WebSocket

These are the prototype's runtime planes. Section 13 shows how they map onto the production platform: Compute becomes the control plane, and a bank data plane (Edge) and a network intelligence plane are added around it.

The two paths

This is the most important property of the design and the thing to say first when a judge asks about the architecture.

- **The control path** is synchronous and deterministic: transaction in, features computed, rules evaluated, policy gate decided, state written, event pushed to the UI. Target end to end: under 200 ms. No AI anywhere in it.
- **The intelligence path** is asynchronous and best-effort: the analyst requests an explanation or an investigation, the backend assembles a structured evidence bundle, Gemini returns schema-validated JSON, the UI renders it.

If the intelligence path is slow or unavailable, the control path is unaffected. Detection, holds and the audit trail keep working; the analyst loses the narrative, not the protection.

Why a single Cloud Run service

One service hosts the ingest consumer, the REST API, and the Gemini calls. Splitting them would look more enterprise and cost us a week of plumbing for no demonstrable benefit at this scale. The WebSocket gateway is the one separate deployable, because its scaling and connection lifecycle are genuinely different. If asked why not microservices, that is the answer: the boundary follows a real difference in runtime behaviour, not an org chart.

---

2. Data model

Seven shapes. Everything else in the system is a view over these.

```ts
interface Transaction {
  id: string;
  ts: string;                 // ISO 8601, UTC
  sourceAccountId: string;
  destAccountId: string;
  payeeId: string;
  deviceId: string;
  amount: number;             // minor units, SGD cents
  channel: 'APP' | 'IBANKING';
  type: 'TRANSFER' | 'GIRO' | 'STANDING' | 'BILL';
  institutionId: string;
  destInstitutionId: string;
}

interface Account {
  id: string;
  institutionId: string;
  balance: number;
  openedAt: string;
  knownDeviceIds: string[];
  knownPayeeIds: string[];
  coolingOffUntil?: string;   // set by a high-risk account event
  protected: boolean;         // balance >= 50_000_00 at window start
}

interface RiskAssessment {
  transactionId: string;
  score: number;              // 0..100
  band: 'LOW' | 'ELEVATED' | 'HIGH' | 'CRITICAL';
  contributions: Contribution[];
  drain: DrainState;
  evaluatedAt: string;
}

interface Contribution {
  code: string;               // 'RAPID_DRAIN', 'NEW_DEVICE', ...
  label: string;              // human-readable, shown in the UI
  points: number;
  detail: string;             // the actual value that fired it
}

interface DrainState {
  balanceAtWindowStart: number;
  outflow24h: number;
  drainPct: number;
  thresholdMet: boolean;
  excluded: boolean;
  exclusionReason?: 'STANDING' | 'GIRO' | 'BILL_ORG';
}

interface Intervention {
  id: string;
  transactionId: string;
  accountId: string;
  state: 'HELD' | 'RELEASED' | 'REJECTED' | 'CANCELLED';
  holdUntil?: string;
  reasonCodes: string[];
  actor: 'SYSTEM' | 'ANALYST' | 'CUSTOMER';
  actedAt: string;
}
```

```
interface Case {
  id: string;
  accountId: string;
  alertIds: string[];
  evidence: EvidenceBundle;
  aiSummary?: CaseSummary;    // Gemini output, schema-validated
  status: 'OPEN' | 'ESCALATED' | 'CLOSED_FRAUD' | 'CLOSED_LEGITIMATE';
}
```

v1.1 additions — the three platform seams

Added 24 September. These three shapes are what let the prototype grow into the multi-tenant platform in section 13 without a rewrite. They cost almost nothing now and a refactor later.

```
// 1. Canonical transaction event — one per lifecycle step, stored immutably in
transaction_events
interface TransactionEvent {
  schemaVersion: '1.0';
  tenantId: string;                  // institution that contributed the event
  eventId: string;                   // unique per event
  idempotencyKey: string;            // dedupe on retry: same key = same event
  accountSequence: number;           // per account; Pub/Sub ordering key = sourceAccountToken
  transactionId: string;             // stable across the lifecycle: PENDING → SETTLED is ONE
payment
  status: 'PENDING' | 'AUTHORISED' | 'SETTLED' | 'REVERSED' | 'CANCELLED';
  sourceAccountToken: string;        // never a raw account number
  destinationInstitutionId: string;
  destinationAccountToken: string;
  networkReference?: string;         // payment-rail reference, when the rail provides one
  deviceToken?: string;              // network token, never a raw fingerprint
  beneficiaryToken?: string;
  amount: number;                    // minor units
  currency: 'SGD';
  channel: 'APP' | 'IBANKING';
  type: 'TRANSFER' | 'GIRO' | 'STANDING' | 'BILL';
  occurredAt: string;
}

// 2. Account security event — feeds cooling-off and payee/device age from real bank signals
interface AccountSecurityEvent {
  schemaVersion: '1.0';
  tenantId: string;
  eventId: string;
  idempotencyKey: string;
  accountToken: string;
  eventType: 'NEW_DEVICE' | 'TOKEN_ACTIVATED' | 'PAYEE_ADDED' | 'PASSWORD_CHANGED'
           | 'MOBILE_NUMBER_CHANGED' | 'PROFILE_CHANGED';
  deviceToken?: string;
  beneficiaryToken?: string;
  occurredAt: string;
}

// 3. Pre-transaction decision — POST /v1/decision/transactions. Reproducible months later.
interface DecisionResponse {
  decisionId: string;
  tenantId: string;
  transactionId: string;
  idempotencyKey: string;            // echoed; a retry returns this same record
  decision: 'ALLOW' | 'HOLD' | 'REJECT';
  riskScore: number;                 // advisory; the gate, not the score, decides
  policyCodes: string[];             // e.g. ['RAPID_DRAIN']
  reasonCodes: string[];             // e.g. ['SG-RD-50_THRESHOLD_EXCEEDED']
  assessmentId: string;              // RiskAssessment with its contributions
  rulesetVersion: string;            // e.g. '2026.09.24'
  policyVersion: string;             // e.g. 'SG-RD-1.0'
  caseId?: string;
  holdUntil?: string;                // auto-release time for HOLD
  decidedAt: string;
  expiresAt: string;                 // after this the bank must ask again
  traceId: string;                   // joins logs, events and audit
}

// 4. Edge provenance — every graph edge says why it exists and whether it may score
interface EdgeEvidence {
  sourceInstitution: string;
```

```
  evidenceType: 'PAYMENT_EVENT' | 'AUTH_EVENT' | 'NETWORK_DEVICE_RELATIONSHIP' |
'BENEFICIARY_MATCH';
  networkReference?: string;
  observedAt: string;
  tenantScope: 'OWN' | 'NETWORK';
  linkage: 'DIRECT' | 'NETWORK_REFERENCE' | 'NETWORK_TOKEN' | 'CORRELATED' | 'UNOBSERVED';  //
DIRECT = one bank's own record
  scoring: 'CONFIRMED' | 'STRONG' | 'CANDIDATE' | 'NONE';   // see ENTITY_LINK weights
}
```

The mapping from `sourceAccountToken` to a display ID like `ACC-10291` lives in the simulator, standing in for the bank's own vault.

v1.2 — lifecycle events versus financial state

A bank sends PENDING, AUTHORISED and SETTLED for the same S$18,000 payment. Stored as three transactions and summed, that becomes a S$54,000 outflow. So storage splits in two:

Collection	Holds	Written by
`transaction_events`	Every lifecycle event, immutable, unique on `idempotencyKey`	Ingest, append only
`transactions`	One document per `transactionId`: current `status` and `effectiveAmount`	Projection, upsert by `transactionId`

Projection rules: PENDING, AUTHORISED and SETTLED set `effectiveAmount = amount` (counted once, whichever arrives first); REVERSED and CANCELLED set it to 0; an event whose `accountSequence` is older than the stored one updates history only, never the state. The rolling 24h outflow and the drain gate read `transactions.effectiveAmount` only. The rule engine is unchanged; its input is now the projection, not the event log.

`AccountSecurityEvent` writes to `security_events` and updates `AccountState` (`newDeviceAt`, `coolingOffUntil`, payee first-seen times). That makes the cooling-off exclusion and the `NEW_DEVICE` / `NEW_PAYEE` signals a real integration capability rather than something only the simulator can produce.

Rules that hold everywhere

- **Money is integer minor units.** No floating-point currency, anywhere. The S$50,000 threshold is `5_000_000`.
- **Time is UTC ISO 8601 on the wire**, localised only at render.
- **`Contribution[]` is the single source of truth for the score.** The score is the sum of its `points`. The UI never recomputes it and Gemini never invents one. This is what makes the attribution panel honest.
- **Interventions are append-only.** A release does not mutate the hold; it adds a record. The audit trail is the point.

Storage split

Data	Store	Why
Account hot state, open alerts, active holds	MongoDB Atlas	Low-latency reads and writes on the control path; change streams push straight to the WebSocket gateway
Transaction history, entity nodes and edges	MongoDB Atlas	One aggregation pipeline for the 24-hour rolling window; `$graphLookup` for two-hop traversal

Data	Store	Why
Session and UI state	Client	Never leaves the browser

One store, not two. The `EvidenceBundle` is a nested document by nature — account profile, assessment with its contributions array, drain state, histories, graph neighbourhood. In a relational store that is eight joins to assemble the single object Gemini receives; in MongoDB it is one read.

---

3. Risk engine

Deterministic, additive, and fully explainable by construction. Given the same transaction and the same account state, it produces the same score and the same attribution, every time.

Feature set and weights

Code	Signal	Max points	Fires when
`RAPID_DRAIN`	Proportion of balance moved in 24h	30	Scales from 0 at 25% drain to 30 at 50%+
`NEW_DEVICE`	Device not in `knownDeviceIds`	21	Binary; 21 if device first seen under 24h ago
`NEW_PAYEE`	Payee not in `knownPayeeIds`	16	Binary; 16 if payee added under 24h ago, 8 if under 7 days
`VELOCITY`	Transactions per minute vs account baseline	12	Scales above 3x the account's rolling baseline
`ENTITY_LINK`	Destination linked to flagged entities	9	Per flagged entity within 2 hops: 3 points via confirmed links (DIRECT, NETWORK_REFERENCE), 2 via NETWORK_TOKEN, 0 via CORRELATED or UNOBSERVED (shown in the graph, never scored). Capped at 9
`BEHAVIOUR_DEV`	Amount vs account's 90-day distribution	7	Scales above the 95th percentile
`COOLING_OFF`	Account within an active cooling-off window	5	Binary

Maximum 100. The demo scenario lands at 95 — that is what these weights produce at 65.4% drain, not a number chosen first and reverse-engineered. A reviewer who sums the table gets the same figure the UI shows.

Bands

Score	Band	UI treatment
0–39	LOW	No alert
40–64	ELEVATED	Queued, amber
65–84	HIGH	Queued, top of list, orange

Score	Band	UI treatment
85–100	CRITICAL	Promoted, red, audible cue in the demo

Scoring

```
function assess(tx: Transaction, acct: Account, ctx: WindowContext): RiskAssessment {
  const contributions: Contribution[] = [];

  for (const rule of RULES) {
    const c = rule.evaluate(tx, acct, ctx);
    if (c && c.points > 0) contributions.push(c);
  }

  const score = Math.min(100, contributions.reduce((s, c) => s + c.points, 0));

  return {
    transactionId: tx.id,
    score,
    band: bandFor(score),
    contributions: contributions.sort((a, b) => b.points - a.points),
    drain: ctx.drain,
    evaluatedAt: new Date().toISOString(),
  };
}
```

Each rule is a pure function returning a `Contribution` or null, with its own unit test asserting exact point values at boundary inputs. Seven rules, seven test files. This is a morning's work and it is what the 40% technical merit weight is actually measuring.

Attribution

The "Why did you stop this transaction?" panel renders `contributions` directly. No transformation, no inference, no second source. Gemini receives the same array and is instructed to explain it in prose — it is explicitly forbidden from adding, removing or re-weighting a contribution. If the model's prose disagrees with the numbers, the numbers are what the UI shows.

Where the 24-hour window comes from

`WindowContext` is computed per transaction from a MongoDB aggregation over `transactions`, cached on the account document and invalidated on every outflow. The Web Worker maintains an optimistic client-side copy so the drain percentage animates smoothly during the demo, but the server value is authoritative and any divergence resolves to the server on the next event.

---

4. Rapid-drain gate and intervention

The risk score is advisory. The gate is not. They are separate code paths on purpose, and the gate is the one that implements the published control.

The gate

```
function drainGate(tx: Transaction, acct: Account, ctx: WindowContext): DrainState {
  // Exclusions bypass the safeguard entirely
  if (tx.type === 'STANDING') return excluded('STANDING', ctx);
  if (tx.type === 'GIRO')     return excluded('GIRO', ctx);
  if (tx.type === 'BILL' && isDesignatedBillingOrg(tx.payeeId))
    return excluded('BILL_ORG', ctx);

  const balanceAtWindowStart = ctx.balance24hAgo;

  // Only accounts at or above S$50,000 are protected
  if (balanceAtWindowStart < PROTECTED_BALANCE) return notProtected(ctx);

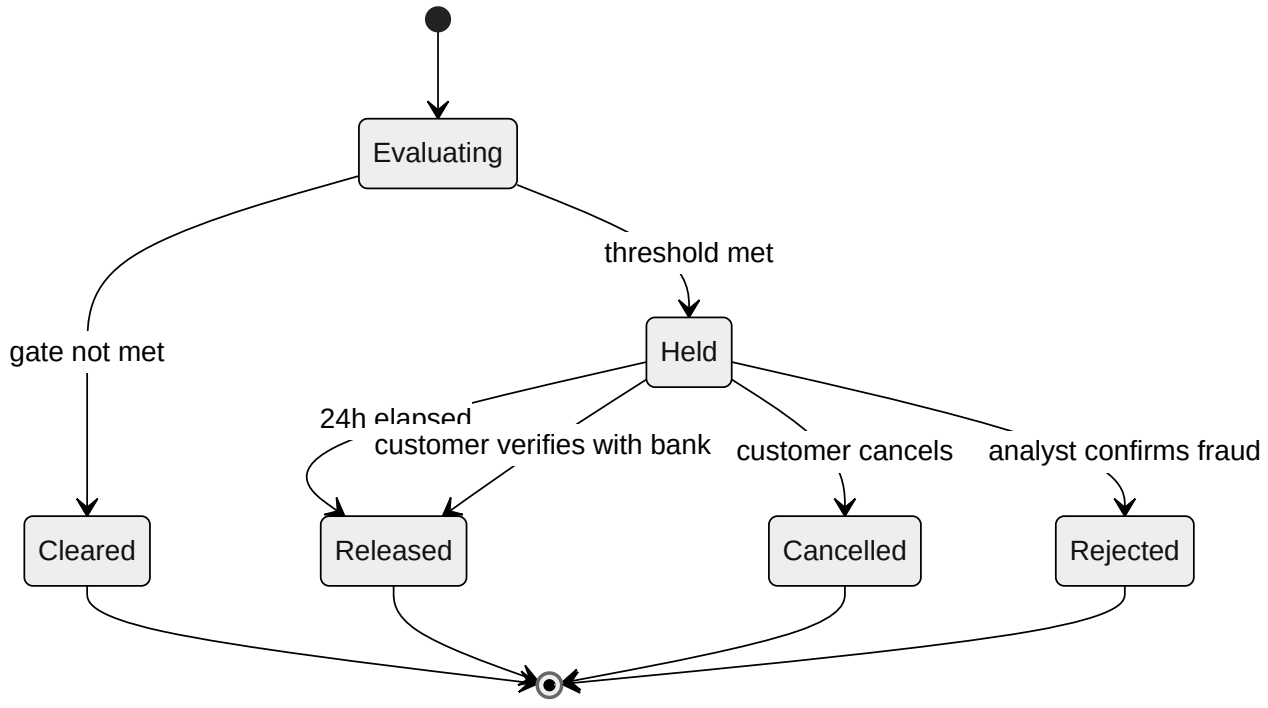
  const outflow24h = ctx.outflow24h + tx.amount;
  const drainPct = (outflow24h / balanceAtWindowStart) * 100;

  return {
    balanceAtWindowStart,
    outflow24h,
    drainPct,
    thresholdMet: drainPct > 50,
    excluded: false,
  };
}
```

`PROTECTED_BALANCE = 5_000_000` (S$50,000 in cents). The comparison is strictly greater than 50%, matching the published wording.

When `thresholdMet` is true, the transaction **and all subsequent outflows on that account** enter the hold state. That cascade is part of the control, not an embellishment, and it is what makes the kill-switch behaviour in the UI accurate rather than invented.

Intervention state machine



Transition	Actor	Notes
Evaluating → Held	SYSTEM	Automatic, immediate, with customer notification fired
Held → Released (timeout)	SYSTEM	Auto-release after the 24-hour cooling period
Held → Released (verified)	CUSTOMER	Expedited release after verification
Held → Cancelled	CUSTOMER	Customer identifies the transfer as fraudulent
Held → Rejected	ANALYST	Escalation only: an analyst can confirm fraud and reject, never release a policy hold without customer verification. Always recorded with the analyst identity

In the demo, the 24-hour timer is compressed to 24 seconds behind a feature flag. The flag is visible in the UI as "demo time ×3600" so nobody can accuse us of misrepresenting the control.

Notification

On entering Held, the system emits a customer notification event carrying the transaction, the reason, and the available actions. In the prototype this renders as a mock mobile-banking push in a corner panel — a small thing that makes the demo feel like a complete loop rather than an internal tool.

Audit trail

Every transition writes an `Intervention` record with actor, timestamp, reason codes, and the `RiskAssessment` snapshot that justified it. The case report is generated from this log, which is why the report can be trusted: it is a rendering of recorded facts, not a reconstruction.

---

5. Frontend architecture

Angular 20, standalone components, no NgModules.

```
src/app/
 ├── core/                 # interceptors, ws client, config, guards
 ├── shared/               # presentational components, pipes, directives
 ├── features/
 │    ├── command-center/
 │    ├── live-transactions/
 │    ├── fraud-alerts/
 │    ├── investigation/
 │    ├── network-analysis/
 │    └── reports/
 ├── state/
 │    ├── alerts/
 │    ├── transactions/
 │    ├── investigation/
 │    └── session/
 └── workers/
      └── transaction.worker.ts
```

State: three tools, three jobs

Using all three is only defensible if each has a boundary you can state in one sentence. Ours:

Tool	Owns	Example
RxJS	Event streams in motion	The WebSocket feed, buffering, worker round-trips
NgRx	Domain state with history and multi-surface reads	Alerts, cases, interventions — anything an action mutates and three components read
Signals	Local UI state	Selected node, panel open, filter chips, sort order

Rule of thumb for reviews: if it survives a route change and another feature reads it, NgRx. If it dies with the component, a Signal. If it arrives over time, RxJS.

The stream layer

```
this.ws.transactions$
  .pipe(
    bufferTime(250),
    filter(batch => batch.length > 0),
    takeUntilDestroyed(),
  )
  .subscribe(batch => this.worker.postMessage({ kind: 'BATCH', batch }));
```

250 ms buffering is the whole trick. At 40 events per second the UI would otherwise do 40 change-detection passes per second; buffered, it does four. The tape still looks continuous because the render animates the batch.

Web Worker contract

The worker exists because there is genuine per-event computation, not because workers are fashionable. It owns:

- Normalisation of the raw event shape
- Rolling transaction velocity per account
- Optimistic 24-hour cumulative outflow and drain percentage
- Payee and device frequency counters
- Diffing: which accounts actually changed in this batch

```
// main -> worker
type WorkerIn =
  | { kind: 'BATCH'; batch: Transaction[] }
  | { kind: 'SNAPSHOT'; accounts: AccountSnapshot[] };  // server truth, resets drift

// worker -> main
type WorkerOut =
  | { kind: 'ACCOUNT_DELTA'; deltas: AccountDelta[] }   // only what changed
  | { kind: 'TAPE'; rows: TapeRow[] };                  // capped at 200 rows
```

The worker sends deltas, never full state. The main thread dispatches one NgRx action per batch, not per transaction. Everything on the main thread is `OnPush`, and the tape is virtualised with a hard cap of 200 rendered rows.

Performance targets

Metric	Target
Sustained event rate without frame drop	50/sec
Main-thread work per batch	Under 8 ms
Time from WebSocket message to visible row	Under 300 ms
Change-detection passes per second	4

These are measured, not asserted. A performance panel behind a key combination shows them live — useful in Q&A when a judge asks whether it really runs in real time.

---

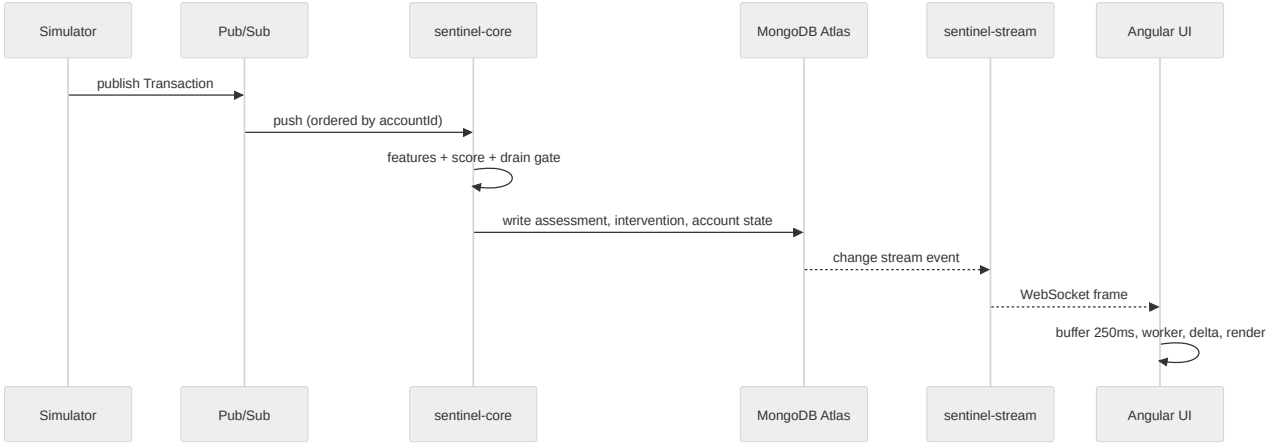
6. Backend and Google Cloud

Deployables

Service	Runtime	Scaling	Purpose
`sentinel-simulator`	Cloud Run job	1 instance	Emits transaction events into Pub/Sub
`sentinel-core`	Cloud Run service	min 1, max 10	Pub/Sub push consumer, risk engine, REST API, Vertex AI calls
`sentinel-stream`	Cloud Run service	min 1, max 3	WebSocket gateway, session-affinity enabled
`sentinel-ui`	Cloud Run service	min 1	Angular SPA, served from a static container

`min instances = 1` on the three services — `sentinel-core`, `sentinel-stream`, `sentinel-ui`. Cold starts are the most common way a hackathon demo dies in the first fifteen seconds, and keeping three instances warm for four weeks costs little next to that risk. `sentinel-simulator` is a **Cloud Run Job**, run on demand per scenario; jobs have no minimum instances and none is needed.

Event flow



Ordering key is `accountId`. Rapid-drain arithmetic is order-dependent: two transfers processed out of order produce a different cumulative outflow at the moment of evaluation. Pub/Sub ordering keys are enabled for exactly this reason, and it is worth saying so if asked — it demonstrates we thought about correctness under concurrency.

Cloud Run WebSockets

Cloud Run supports WebSockets, with two constraints we design around:

- **Request timeout caps connection lifetime.** Set to 60 minutes; the client reconnects with exponential backoff and resumes from a server-issued global stream cursor (the per-account sequence orders one account; it cannot resume a multi-account tape).
- **Session affinity is best-effort.** So the gateway holds no per-connection state that cannot be rebuilt: on reconnect the client requests a snapshot, the server replays current account state, and the stream resumes.

This matters for the demo. A dropped connection mid-pitch must recover invisibly within two seconds.

MongoDB collections

```
accounts             hot state, cached drain window, known devices and payees
transaction_events   immutable lifecycle log, unique {idempotencyKey: 1}
transactions         projection: one doc per transactionId, effectiveAmount, indexed
{sourceAccountId: 1, ts: -1}
security_events      AccountSecurityEvent log, feeds cooling-off and device/payee age
decisions            DecisionResponse records, unique {idempotencyKey: 1}
assessments          RiskAssessment including contributions[]
interventions        append-only audit log — change stream watched
alerts               open alert queue — change stream watched
cases                investigation state and Gemini output
entities             graph nodes
entity_edges         graph edges with EdgeEvidence, indexed {from: 1} and {to: 1}
```

Three schema rules hold everywhere:

- **Money is integer minor units.** SGD cents, never a float. `PROTECTED_BALANCE` is `5_000_000`.
- **`interventions` is append-only.** A release inserts a record; it never mutates the hold.
- **`contributions[]` is written once by the engine and never updated.** No code path may modify a stored contribution. That is the database-level expression of "Gemini cannot re-weight the score".

Aggregations

The 24-hour rolling outflow is a `$match` on account and timestamp, `$group` to sum, `$merge` into the cached window on the account document — invalidated on every outflow. The two-hop neighbourhood behind the entity graph is a `$graphLookup` with `maxDepth: 1`, capped at 40 nodes and ranked by edge weight so the cap drops the least interesting edges first. Both are cached per account or per root node for the session, so the control path never waits on an aggregation.

Environment and secrets

All model identifiers, thresholds and feature flags live in environment config on the Cloud Run service, not in code. Vertex AI is called with the service account's own credentials; there are no API keys in the repository, and the README says so explicitly because judges do look.

7. Gemini integration

Two tiers, routed by job

Tier	Used for	Latency budget
Fast	Risk explanation, alert enrichment, entity summary, next-step suggestion	Under 2 s, streamed
Deep	Case summary, multi-source evidence correlation, final case report	Under 15 s, progress shown

Model identifiers are **environment config, never literals in code**, behind a `ModelRouter` with a single `resolve(job)` method. The current working assumption is a current-generation Flash model for the fast tier and a current-generation Pro reasoning model for the deep tier. Verify the exact available model IDs against the Vertex AI models page on **5 October**, and again in the week before the finale — a retired model ID on stage is an avoidable, fatal, and entirely predictable failure.

Evidence bundle

Gemini never sees a raw database dump and never sees a free-form prompt assembled by string concatenation. It receives one typed object:

```
interface EvidenceBundle {
  account: AccountProfile;          // no names, no identifiers beyond synthetic ids
  assessment: RiskAssessment;       // including contributions[]
  drain: DrainState;
  recentTransactions: Transaction[];   // last 50, capped
  deviceHistory: DeviceEvent[];
  payeeHistory: PayeeEvent[];
  priorAlerts: AlertSummary[];
  interventions: Intervention[];
  neighbourhood: GraphNeighbourhood;   // 2 hops, capped at 40 nodes
}
```

Capping matters: it bounds cost, bounds latency, and bounds the model's opportunity to wander.

Output contracts

Every call uses structured output with a response schema. Unparseable or schema-invalid responses are retried once, then the UI falls back to rendering the deterministic attribution alone — which is always available, because the rule engine produced it.

```
interface CaseSummary {
  hypothesis: string;                       // <= 300 chars
  confidence: 'LOW' | 'MEDIUM' | 'HIGH';
  evidence: Array<{
    claim: string;
    sourceField: string;                    // must name a field in the bundle
    value: string;
  }>;
  recommendedActions: string[];             // <= 5
  escalate: boolean;
}
```

`sourceField` is the guardrail. Every claim must cite the bundle field it came from, and the backend validates that each `sourceField` actually exists in the bundle it sent. A claim that cites nothing is dropped before it reaches the UI. This is cheap to implement and it is the single most convincing thing we can show a BFSI judge about hallucination control.

System prompt constraints

The system instruction states plainly, and we quote it in the submission:

- You are explaining a decision that has already been made by a deterministic rule engine
- You may not add, remove, or re-weight any risk contribution
- You may not assert a fact that is not present in the evidence bundle
- You may not recommend releasing a held transaction
- Where evidence is insufficient, say so rather than inferring

That fourth constraint is deliberate and worth stating out loud in the pitch: **the model is structurally incapable of recommending that money be let go.** It can only explain, correlate and escalate.

Caching and the demo

Explanations are cached by `assessmentId`. The demo account's explanation is pre-warmed before the pitch begins, so scene 7 renders instantly regardless of network conditions in the venue. This is stated in the README rather than hidden.

---

8. Entity graph

Schema

Nodes and edges, both flat, both in MongoDB — the `entities` and `entity_edges` collections.

```
type NodeKind = 'ACCOUNT' | 'PAYEE' | 'DEVICE' | 'INSTITUTION';

interface GraphNode {
  id: string;
  kind: NodeKind;
  institutionId?: string;
  flagged: boolean;
  riskIndicators: string[];   // 'RAPID_MOVEMENT', 'MULTI_BENEFICIARY', ...
  txCount: number;
}

interface GraphEdge {
  from: string;
  to: string;
  kind: 'TRANSFERRED_TO' | 'USED_DEVICE' | 'PAYEE_OF' | 'HELD_AT';
  weight: number;             // total value moved, or event count
  firstSeen: string;
  lastSeen: string;
}
```

Traversal

Two hops from the selected node, capped at 40 nodes and 120 edges, ranked by edge weight so the cap drops the least interesting edges first. A single `$graphLookup` with `maxDepth: 1`, cached per root node for the

session.

Two hops is not a limitation we apologise for — it is the depth at which a mule fan-out becomes visible and beyond which the picture turns into hairball. Say that if asked.

Cross-institution synthesis

The demo network is generated, not scraped, and its structure is designed to make the pattern legible:

Layer	Count	Role
Institutions	3	Bank A (ours), Bank B, Bank C
Victim accounts	12	Across all three, normal behaviour
Mule accounts	7	Receive, then forward within minutes
Shared devices	2	The fingerprint linking mules across institutions
Consolidation accounts	1	Where the funds converge

The shared device fingerprint is what makes the graph tell a story rather than show a mess. It is the edge that no single bank could see today, which is the whole point of the COSMIC-inspired framing.

Rendering

Force-directed layout, canvas not SVG above 60 nodes. The layout simulation runs in its own Web Worker so dragging a node never stutters the transaction tape behind it.

Interaction contract:

- Click a node → right rail shows connected accounts, institutions, transaction count, risk indicators
- Hover an edge → value moved and time window
- Flagged nodes carry a persistent visual mark, not just a colour, so the graph survives a projector with poor colour reproduction
- One control: hop depth, 1 or 2

No zoom-to-fit animations, no physics tuning sliders, no minimap. Every control we do not build is a control that cannot fail on stage.

---

9. Synthetic data and scenarios

All data is generated. No real, scraped, or anonymised customer data touches this project, and the README states that in the first paragraph.

Generators

Generator	Produces
`baseline`	Normal traffic: salary credits, bill payments, transfers to known payees, plausible diurnal rhythm
`noise`	Low-level oddities that score ELEVATED but never trip the gate — proves the system is not just alarming on everything
`takeover`	The phishing-driven account takeover chain

Generator	Produces
`mule-network`	The cross-institution fan-out and consolidation structure

The `noise` generator matters more than it sounds. A demo where every transaction is fraud proves nothing about discrimination. The judges should see the system *not* alerting most of the time.

The scenario engine

```
interface Scenario {
  id: string;
  seed: number;             // deterministic PRNG, no Math.random anywhere
  timeline: ScenarioStep[];
}

interface ScenarioStep {
  atMs: number;             // offset from scenario start
  action: 'LOGIN_NEW_DEVICE' | 'ADD_PAYEE' | 'TRANSFER' | 'NOOP';
  params: Record<string, unknown>;
}
```

Every random draw comes from a seeded PRNG. The same seed produces the same stream, the same scores, the same hold, every run. This is what makes the demo safe and what makes the regression test possible — they are the same mechanism.

The demo scenario: `DEMO_TAKEOVER_01`

Offset	Event	Effect
0–30 s	Baseline traffic across 12 accounts	Everything LOW
30 s	Login on unseen device, ACC-10291	`NEW_DEVICE` fires, score 21
34 s	New payee created	`NEW_PAYEE` fires, score 37
41 s	Transfer S$2,400	Velocity baseline begins to move
42 s	Transfer S$4,900	Drain 29.4%, RAPID_DRAIN and BEHAVIOUR_DEV fire, score 49, ELEVATED
43 s	Transfer S$7,200	`VELOCITY` fires at 43.3% drain, score 75, HIGH
45 s	Transfer S$11,500	Drain 65.4%, gate trips, score 95, CRITICAL, HOLD
46 s	Customer notification	Mock push appears
48 s+	Analyst investigates	Evidence bundle, Gemini summary, graph

Account opens with S$52,000. Total moved: S$34,000 over the 24-hour window including prior activity. Drain: 65.4%. These are the numbers in the master document's demo narrative and they must match exactly — one shared constants file, imported by both the generator and the documentation build.

Determinism as a test

CI runs `DEMO_TAKEOVER_01` headless and asserts: score reaches 95, `thresholdMet` becomes true at step 7, the intervention state is HELD, and `contributions` sums to the displayed score. If that test fails, the demo is broken, and we know before the judges do.

---

10. Budgets

Latency

Path	Target	Hard ceiling
Pub/Sub publish to risk decision	80 ms	200 ms
Risk decision to WebSocket frame	40 ms	120 ms
WebSocket frame to rendered row	300 ms	500 ms
Gemini fast tier, first token	800 ms	2 s
Gemini deep tier, complete	8 s	15 s
Graph query, 2 hops cached	150 ms	600 ms

The control path total — transaction to visible, scored, gated row — must stay under **500 ms**. That is the number we quote on stage.

Cost

Four weeks of build plus the finale. Rough ceiling, to be checked weekly rather than discovered at the end.

Item	Estimate
Cloud Run, 4 services, min-instance 1	Largest single line; keep instances small (1 vCPU / 512 MiB)
Pub/Sub	Negligible at demo volumes
Atlas connection pool	One pool created at module load, never per request
MongoDB Atlas	Start on M0 free tier; index every query path and cache graph lookups. Upgrade to M10 only if change streams or connection limits force it
Vertex AI	Bounded by the evidence bundle caps and explanation caching

Set a billing budget alert at a figure the team agrees on in week one. An exhausted free tier on 17 October is a self-inflicted wound.

Security

It is a prototype, and we say so — but the things that would embarrass us are cheap to get right:

- No secrets in the repository; Vertex AI called with the service account's own credentials
- No real personal data, stated in the README's first paragraph
- Synthetic identifiers only; no names, NRICs, card numbers or account numbers that resemble real ones
- Single demo login, clearly labelled as a demo, not presented as an auth system

- CORS restricted to the deployed UI origin
- Analyst actions recorded with an actor identity, because the audit trail is a product feature

What we explicitly do not claim: production-grade authentication, authorisation, encryption at rest beyond platform defaults, or penetration testing. Claiming those would invite a question we cannot answer well. The honest framing — *this is a functional prototype of a control, with the security posture of a prototype* — costs nothing and is unassailable.

---

11. Repository and build order

Layout

```
sentinel-ai/
 ├── apps/
 │    ├── ui/              # Angular 20 SPA
 │    ├── core/            # Cloud Run service: ingest, risk, API, Gemini
 │    ├── stream/          # WebSocket gateway
 │    └── simulator/       # Cloud Run job
 ├── packages/
 │    ├── domain/          # shared types, constants, thresholds
 │    ├── rules/           # the seven rules + tests, no I/O
 │    └── scenarios/       # generators, seeded PRNG, DEMO_TAKEOVER_01
 ├── infra/               # deploy scripts, Cloud Run config
 └── docs/                # architecture diagram, submission material
```

`packages/domain` holds the thresholds and the demo constants, and both the runtime and the documentation build import from it. That is how the numbers in the pitch stay identical to the numbers on screen.

`packages/rules` has no I/O and no framework dependency. It is pure functions and tests, runnable in Node and in the browser Worker — which is what lets the client compute optimistic scores without a second implementation.

Build order

Strictly sequential. Each step is demoable before the next begins.

Order	Deliverable	Depends on	Target
1	`packages/domain` types and constants	—	22 Sep
2	Simulator emitting to Pub/Sub	1	24 Sep
3	`core` consuming and persisting	2	26 Sep
4	`stream` gateway and Angular shell receiving	3	28 Sep
5	**Transaction visible in deployed UI**	4	**30 Sep**
6	Rules package and scoring, with tests	1	2 Oct
7	Drain gate and intervention state machine	6	4 Oct
8	Worker, buffering, tape, alert queue	5, 6	5 Oct
9	Gemini explanation with schema validation	6	6 Oct

Order	Deliverable	Depends on	Target
10	Investigation workspace and case summary	9	7 Oct
11	Entity graph, data and rendering	3	8 Oct
12	Case report generation	10	9 Oct
13	Demo scenario locked, CI assertion green	all	12 Oct
14	Polish, submission package	all	15 Oct

Step 5 is the go/no-go. If a transaction is not visible in a deployed UI by 30 September, scope comes out that day — the entity graph first, since it is the most self-contained module to drop.

v1.1 changes to the order. The `TransactionEvent` envelope and the decision route join step 1–5 work, before the first stream is deployed — changing the event shape after the UI consumes it costs a day. `EdgeEvidence` lands with the entity graph step. The Gemini-outage toggle and tenant lens join the polish window, in that order.

v1.2 changes to the order. `transaction_events` plus the `transactions` projection land with the envelope, before any drain maths is written against them — the headless scenario test gains one assertion: a PENDING then SETTLED pair counts once. `security_events` lands with the rule engine. The versioned `DecisionResponse` and the linkage-weighted `ENTITY_LINK` rule are same-day changes, not new steps.

CI

One workflow: typecheck, unit tests on `packages/rules`, the headless `DEMO_TAKEOVER_01` assertion, build all four apps, deploy on green from main. Nothing elaborate. The scenario assertion is the test that matters.

---

12. Open decisions

Eight things to settle by **23 September**. Each has a recommendation, so the default is to accept unless someone objects.

#	Decision	Recommendation
1	NgRx, or Signals-only store	NgRx for alerts, cases and interventions. If the team has no NgRx experience, drop it entirely and use a signal store — half-learned NgRx in week two is worse than neither
2	Graph rendering library	Whatever the frontend owner already knows. This is not the place to learn a new library
3	MongoDB Atlas cluster tier	Start on M0 free tier. Upgrade to M10 only if change streams or connection limits force it — one dropdown, small cost
4	Aggregation performance for the rolling window	Measure it. If the two-hop graph lookup or the rolling-window pipeline misses its latency budget, cache harder or compute in-memory in `core`. Do not add a second analytical store.
5	Demo time compression factor	×3600, so 24 hours reads as 24 seconds, labelled visibly in the UI
6	Voice assistant, in or out	Out of the MVP. Revisit only after 12 October, and only for the finale, not the submission

#	Decision	Recommendation
7	Who owns compliance wording review	One named person owns every mention of SRF and COSMIC across the UI, README, proposal and video
8	Fallback if Vertex AI quota is denied on the project	Request quota in week one so this is discovered early, not on 14 October
9	Decision API in the prototype: synchronous REST, or Pub/Sub only	Synchronous REST on sentinel-core for the simulator's pre-commit call, then publish the same event to Pub/Sub. One extra route, and it is the answer to the bank CTO's first question
10	Tenant lens, in or out	Stretch. Build only if the 7 October walkthrough passes; client-side projection over synthetic data, no auth changes

Two things to verify, not decide

1. **The submission deadline.** Official site, week of 28 September. Third-party listings disagree.
2. **Available Gemini model IDs on Vertex AI.** Check 5 October, and again the week before the finale.

The one architectural commitment

Everything else here is negotiable. This is not:

> The policy gate never calls a language model, and the language model never returns a decision. They are parallel paths that meet only at the analyst's screen.

If a proposed change breaks that property, it is the wrong change — regardless of what it adds.

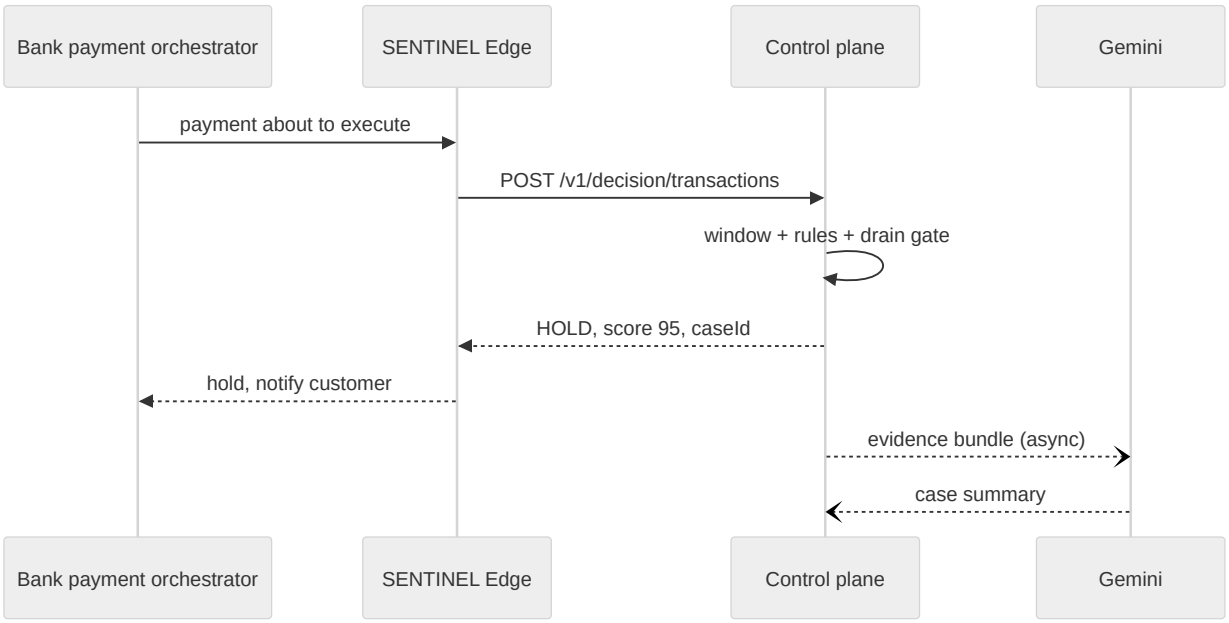
---

13. Production architecture

The production platform wraps today's single service in four planes. The prototype's compute plane becomes the control plane; the other three are added around it. This section describes; only the seams in section 2 (v1.1 and v1.2 additions) are built for the submission.

Plane	Responsibility	Lives where
Bank data plane (SENTINEL Edge)	Connect to bank systems, normalise to `TransactionEvent`, tokenise identifiers, enforce outbound data policy, return decisions	Bank's approved integration zone
Control plane	Tenants, policies, risk engine, intervention, audit, connector management	SENTINEL cloud
Network intelligence plane	Tokenised cross-bank projection, correlation, network risk	SENTINEL cloud, permitted fields only
Analyst experience plane	Per-tenant control tower, investigation, trace, Gemini	Angular, per-tenant tokens

Decision flow



The synchronous leg never waits on Gemini. The latency budget in section 10 (500 ms for the control path) is the contract a bank would sign against.

Integration modes

Mode	Transport	Purpose
Pre-transaction decision	REST, `POST /v1/decision/transactions`	Prevention
Event stream	Bank Kafka / MQ → Edge → Pub/Sub	Detection, graph building
Webhook / callback	Signed HTTPS both ways	Intervention lifecycle
Batch / SFTP	File drop → Cloud Storage → reconcile job	Backfill, reconciliation

One Connector SDK with REST, event and file adapters. No per-bank services.

Cross-bank linkage

Each bank contributes only its own legs. The network plane joins legs in this order of strength, and records which one it used as `EdgeEvidence.linkage`:

1. `NETWORK_REFERENCE` — the same payment-rail reference appears in A→B and in B's inbound record.
2. `NETWORK_TOKEN` — a pseudonymous account token issued under a trusted arrangement, so banks match subjects without exposing account numbers.
3. `CORRELATED` — beneficiary or payment correlation; shown with lower confidence.
4. `UNOBSERVED` — no valid linkage. The trace ends here and the UI says so.

Trace responses carry `observability: { complete: false, boundary: 'BANK_C' }` so the UI never implies a complete path.

Graph store evolution

D3 or Canvas is the renderer, not the store. The prototype keeps MongoDB `entities` / `entity_edges` with `$graphLookup`. At network scale the graph becomes a projection into a dedicated store; Spanner Graph supports ISO GQL property-graph queries and lists fraud detection as a use case. Not adopted for the submission.

---

14. Tenancy, reliability and failure

Tenant configuration

One codebase, one deployment, many tenants. Angular reads design tokens and policy from the tenant record; nothing is forked per bank.

```
interface TenantConfig {
  tenantId: string;
  branding: { logo: string; primary: string; accent: string };
  policies: { rapidDrain: { protectedBalance: number; gatePct: number; holdHours: number } };
  identity: { provider: 'SAML' | 'OIDC'; issuer: string };
  retentionDays: number;
  connectors: ('DECISION_API' | 'EVENT_STREAM' | 'WEBHOOK' | 'BATCH')[];
  fallback: 'ALLOW' | 'QUEUE' | 'LOCAL_RULE';   // set by the bank, used when SENTINEL is
unreachable
}
```

Per-tenant identity maps to Identity Platform multi-tenancy, where each tenant holds its own identity providers (SAML or OIDC), settings and users.

Field-level visibility

`tenantId` filtering is not enough. The API projects every node through a visibility rule before it leaves the server:

Field	Own tenant	Other tenant (network view)
Account display	Full internal reference	`••••4921`
Customer identity	Available to authorised roles	Never
Transaction history	Full	Count withheld; relationship only
Risk indicator	Full score and contributions	Band only (e.g. HIGH)
Path position	Yes	Yes (hop count)

The prototype's tenant lens (stretch) applies this table client-side over synthetic data, to show the behaviour. Production enforces it server-side.

Event reliability

Banks send events late, out of order and twice. The envelope carries `eventId`, `idempotencyKey` and `sequence`; the pipeline adds:

- **Dedupe** on `idempotencyKey` (unique index on `transaction_events`); a retried decision request returns the stored `DecisionResponse`. Lifecycle updates change the `transactions` projection, never add to it.

- **Ordering** with the Pub/Sub ordering key per account token, which the drain arithmetic already depends on.
- **Replay and dead-letter** — a DLQ subscription for events that fail schema validation, replayable after a fix.
- **Reconciliation** — the batch mode compares daily files against what the stream delivered.

The prototype builds dedupe and ordering. DLQ and reconciliation are described only.

Failure modes

Failure	What happens to the payment
Gemini unavailable	Nothing. Decisions, holds and audit continue; the analyst loses the narrative
Control plane unreachable from Edge	Tenant's `fallback` applies — the bank chooses, not SENTINEL
Event stream delayed	Decisions still made on the synchronous path; graph catches up

The prototype demonstrates the first row live with a Gemini-outage toggle.

Production security (described, not deployed)

Private or controlled connectivity from each bank, mTLS at the edge, API Gateway with OAuth2/OIDC and schema validation, per-service identities on Cloud Run, Secret Manager, encryption with customer-managed keys, RBAC plus attribute-based rules for the visibility table, immutable intervention history, retention per tenant, break-glass access with audit. Singapore's PDPA purpose-limitation duty and MAS technology-risk expectations apply to any real deployment; the submission uses synthetic data only.

Audit that survives a regulator's question

"Append-only" in MongoDB is an application promise, not an immutable record. Production adds a second tier: every intervention and decision is copied to an immutable archive (object storage with retention lock and legal hold), separate from the operational database. Each intervention record carries `decisionId`, `transactionId`, `actor`, `timestamp`, `reasonCodes`, `policyVersion`, `rulesetVersion`, `previousState` and `newState`, so "why was this held on 24 September?" is answered by replaying the stored event, account state, ruleset and policy. The prototype writes these fields; the archive tier is described only.

Data separation

Identity data (names, account numbers, national identifiers) stays in the bank or a separate PII vault. The investigation graph holds tokens and provenance; analytics holds aggregates. Three stores, three access policies — the graph can never be used to re-identify a customer on its own. Cross-bank device links exist only as network-authorised device relationships: each bank tokenises through an approved mechanism, and SENTINEL never compares raw fingerprints.